



MAESTRIA EN INTELIGENCIA ARTIFICIAL APLICADA

*Proyecto: Agente IA para gestión defensiva de vulnerabilidades
con inferencia híbrida, RAG y analítica operacional*

EMCALI Cyber 2.0 Híbrido v3

Autores: RUBEN DARIO SABOGAL URBANO

EDWIN PEREZ LOZANO

CRISTIAN CAMILO QUEBRADA

Asesor: CRISTIAN CAMILO URCUQUI

Cali, mayo del 2026

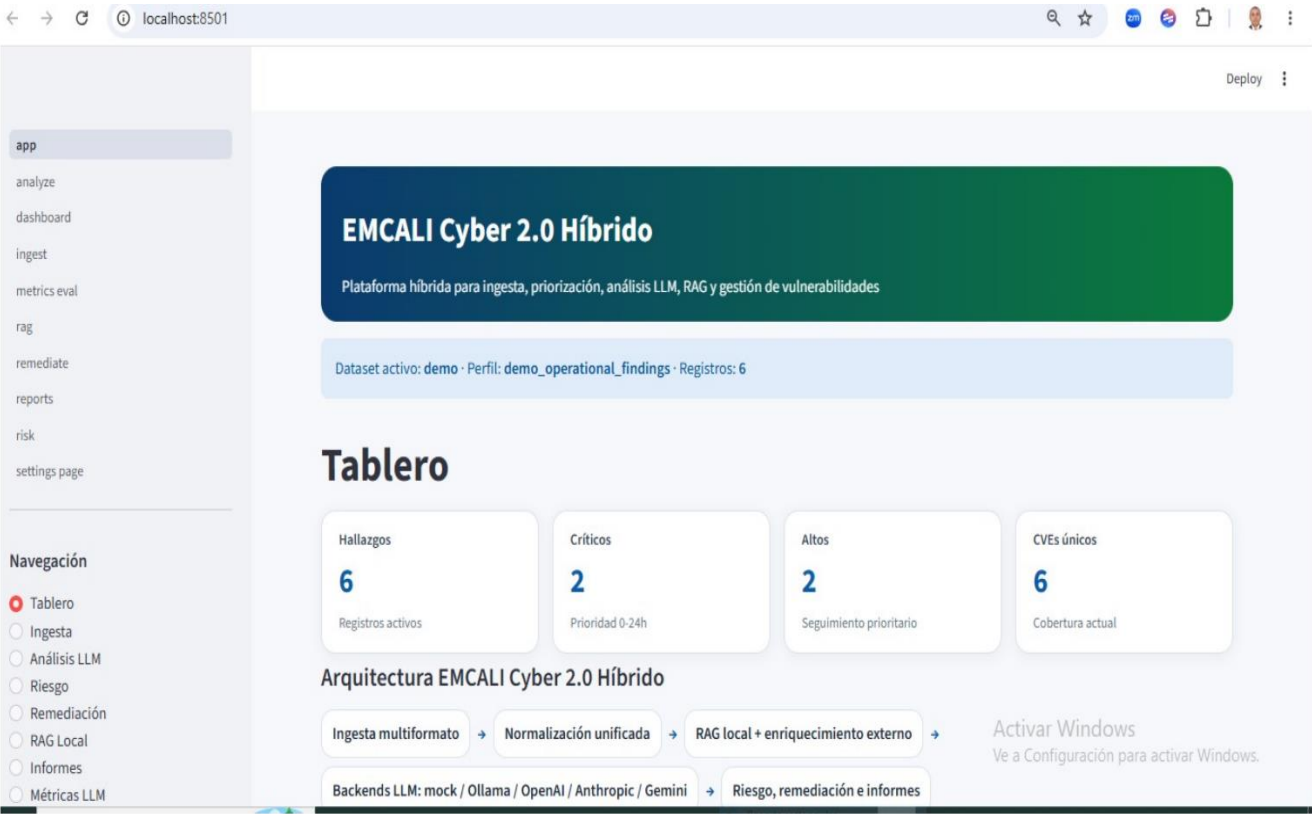
EMCALI Cyber 2.0 Híbrido v3

Informe técnico consolidado de diseño, arquitectura, operación, analítica e implementación

Proyecto: Agente IA para gestión defensiva de vulnerabilidades con inferencia híbrida, RAG y analítica operacional

Autores del proyecto: Ruben Darío Sabogal – Edwin perez Londoño – Cristian Camilo Quebrada

Documento consolidado actualizado con arquitectura Híbrido v3, ingesta multiformato y encadenamiento total por session_state



Resumen ejecutivo

Este informe técnico consolida la evolución del proyecto EMCALI Cyber 2.0 hasta su versión Híbrido v3. La solución implementa una aplicación modular en Streamlit para gestión defensiva de vulnerabilidades, con soporte de ingesta multiformato, normalización heurística, estado compartido de sesión, recuperación aumentada por conocimiento local (RAG), enriquecimiento externo controlado y selección de backend de inferencia entre modos local y cloud. La versión v3 resuelve el principal problema observado en iteraciones anteriores: la falta de encadenamiento completo entre módulos cuando el usuario cargaba nuevos archivos en Ingesta. Ahora el dataset activo se propaga a Tablero, Análisis, Riesgo, Remediación, RAG, Informes y Métricas, lo que permite una operación coherente de extremo a extremo. Además, se incorporan perfiles de entrada para datasets operativos y enriquecidos, incluyendo exportaciones tipo Qualys, Tenable/Nessus, NVD + CISA KEV + EPSS, corpus de CVE y archivos tabulares en CSV, JSON y Excel. El resultado es un prototipo funcional, demostrable y empaquetable para Windows, apto para sustentación técnica y académica.

Abstract

This technical report consolidates the evolution of the EMCALI Cyber 2.0 project up to its Híbrido v3 release. The solution implements a modular Streamlit application for defensive vulnerability management with multi-format ingestion, heuristic normalization, shared session state, retrieval-augmented generation, controlled external enrichment and hybrid backend selection across local and cloud inference. Version v3 solves the key limitation observed in previous iterations: the lack of full chaining between modules after loading new input files in Ingesta. The active dataset now propagates to Dashboard, Analysis, Risk, Remediation, RAG, Reports and Metrics, enabling end-to-end operational consistency. The system also recognizes operational and enriched datasets, including Qualys, Tenable/Nessus, NVD + CISA KEV + EPSS profiles, CVE corpora and tabular files in CSV, JSON and Excel. The result is a functional, demonstrable and Windows-packagable prototype suitable for technical and academic defense.

Contenido sintético

- 1. Contexto y evolución del proyecto
- 2. Objetivo, alcance y principios de diseño
- 3. Arquitectura Híbrido v3 y diagramas nuevos
- 4. Flujo operativo y encadenamiento entre módulos
- 5. Ingesta multiformato y normalización avanzada
- 6. Inteligencia híbrida, RAG y enriquecimiento externo
- 7. Descripción detallada de módulos, analítica y empaquetado
- 8. Evidencia gráfica, resultados, limitaciones y conclusiones
- 9. Anexos técnicos con estructura de código y extractos clave

1. Contexto y evolución del proyecto

EMCALI Cyber 2.0 nace como un prototipo aplicado para asistir la gestión de vulnerabilidades en un entorno corporativo de servicios públicos. Las primeras iteraciones se enfocaron en demostrar la viabilidad del uso de un LLM local, la carga básica de hallazgos desde fuentes como Qualys y Tenable y la generación de resúmenes de riesgo. Posteriormente se agregaron módulos para RAG local, informes, métricas y empaquetado en Windows. La versión Híbrido v3 representa la maduración del diseño: no solo añade backends cloud opcionales y soporte de datasets enriquecidos, sino que corrige la lógica interna para que la ingesta gobierne el conjunto activo de datos consumido por todos los módulos.

La evolución conceptual puede resumirse así:

- De un modelo local aislado a una arquitectura híbrida con ruteo por backend.
- De una carga demo fija a una ingesta multiformato con perfiles heurísticos.
- De módulos parcialmente desacoplados a un encadenamiento unificado por session_state.
- De una visualización estática a un tablero y analítica dinámica dependiente del dataset activo.
- De un prototipo de exploración a un artefacto demostrable con empaquetado Windows y control de configuración.

2. Objetivo, alcance y principios de diseño

El objetivo técnico de la versión Híbrido v3 es proporcionar una plataforma demostrable y extensible para ingestión, evaluación y priorización de vulnerabilidades, integrando analítica de riesgo, RAG local, enriquecimiento externo y selección de modelos de inferencia según capacidad del equipo y disponibilidad de conectividad.

2.1 Alcance funcional

Dimensión	Descripción
Fuentes de entrada	CSV, JSON, Excel, exportaciones tipo Qualys, Tenable/Nessus, datasets NVD+CISA+EPSS y corpus CVE.
Estado operativo	Session_state compartido con findings_df, knowledge_base, analysis_history, reports_history y settings.
Módulos	Tablero, Ingesta, Análisis LLM, Riesgo, Remediación, RAG Local, Informes, Métricas LLM y Configuración.
Backends LLM	mock, ollama, openai, anthropic y gemini.
Salidas	Priorización, matriz de riesgo, recomendaciones, respuesta LLM, informe técnico, métricas y JSON del análisis.
Empaquetado	Código fuente, ZIP portable, ejecutable Windows y Setup cuando NSIS está disponible.

2.2 Principios de diseño

- Multiformato: la aplicación no depende de un único esquema de entrada.
- Encadenamiento por session_state: la ingesta define el dataset activo que consumen los demás módulos.
- Inferencia híbrida: la selección del backend responde al hardware, costo y necesidad de latencia.
- RAG + enriquecimiento externo: el análisis combina evidencia local y contexto CVE adicional.
- Reportabilidad y métricas: el sistema produce evidencia ejecutiva, técnica y de evaluación del agente.

3. Arquitectura de la solución Híbrido v3

La solución se organiza alrededor de tres ideas centrales: (i) un modelo único de arquitectura para el flujo completo de datos, (ii) un diagrama lógico por capas que separa responsabilidades, y (iii) una topología funcional que evidencia el encadenamiento entre ingesta, módulos internos, servicios externos y productos de salida.

EMCALI Cyber 2.0 Híbrido v3 – Modelo Único de Arquitectura

Agente de gestión de vulnerabilidades con inferencia híbrida, RAG y analítica operacional

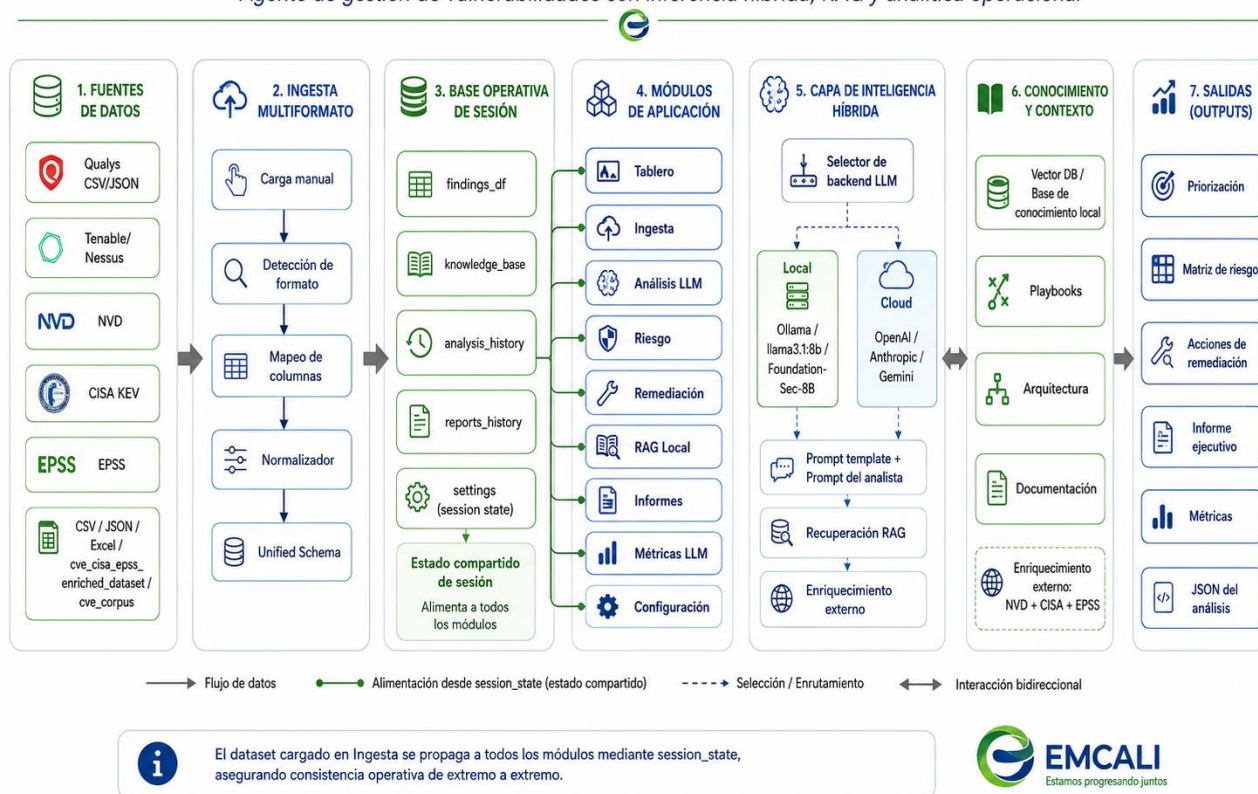


Figura 1. Modelo único de arquitectura de EMCALI Cyber 2.0 Híbrido v3.

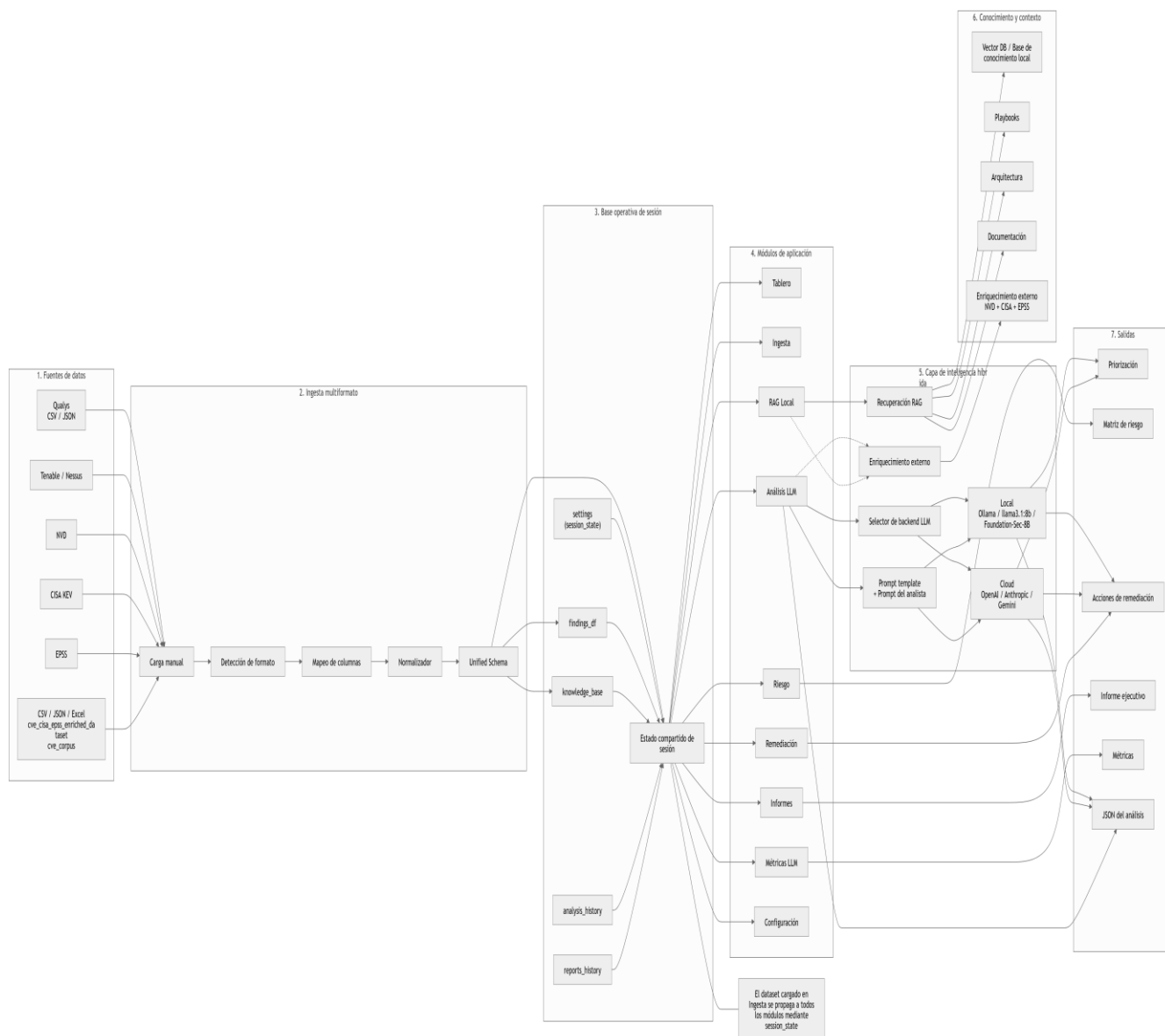


Figura 2. Diseño Modelo único de arquitectura de EMCALI Cyber 2.0 Híbrido v3.

Diagrama lógico por capas – EMCALI Cyber 2.0 Híbrido v3

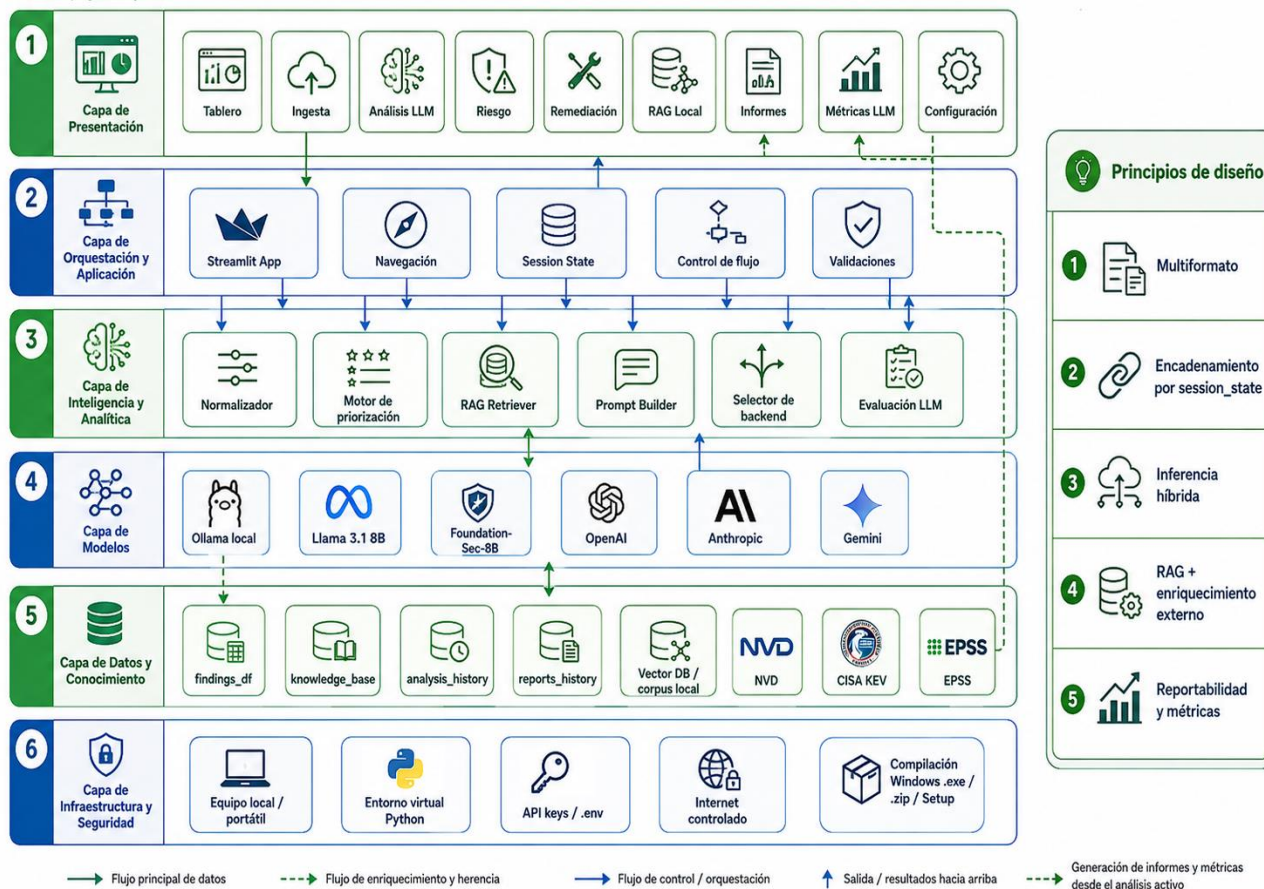


Figura 3. Diagrama lógico por capas.

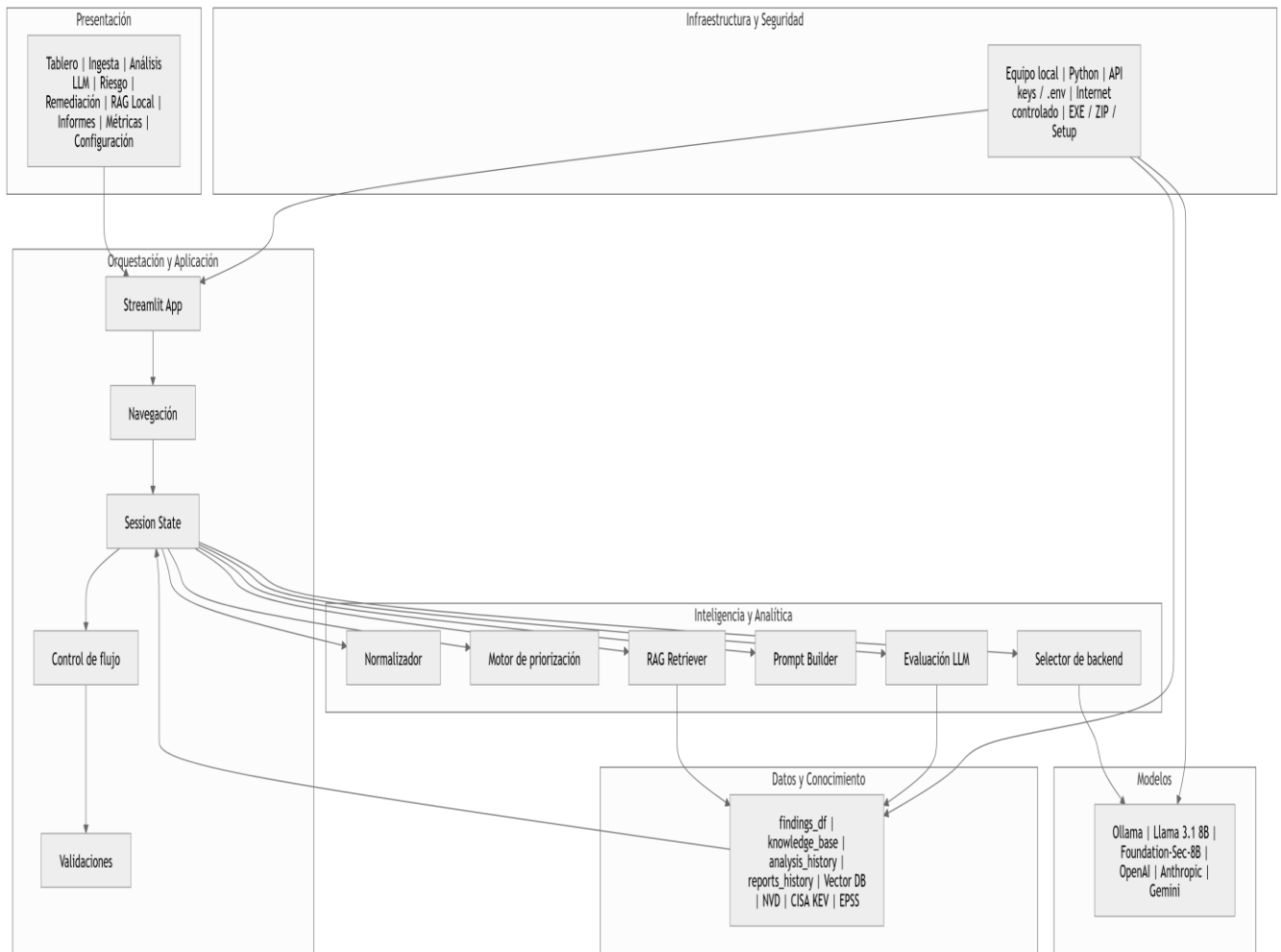


Figura 4. Diseño Diagrama lógico por capas.

• Topología funcional y módulos internos con ingesta de datos •

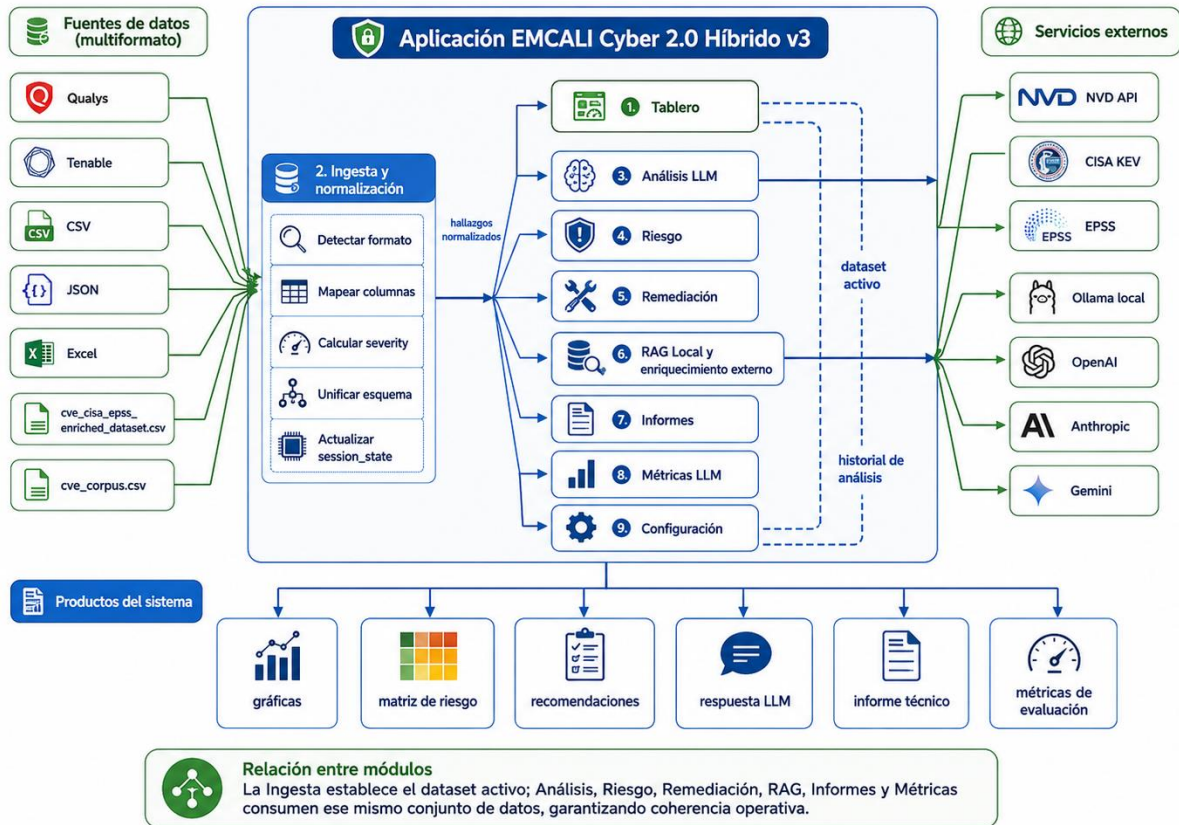


Figura 4. Topología funcional y módulos internos con ingesta de datos.

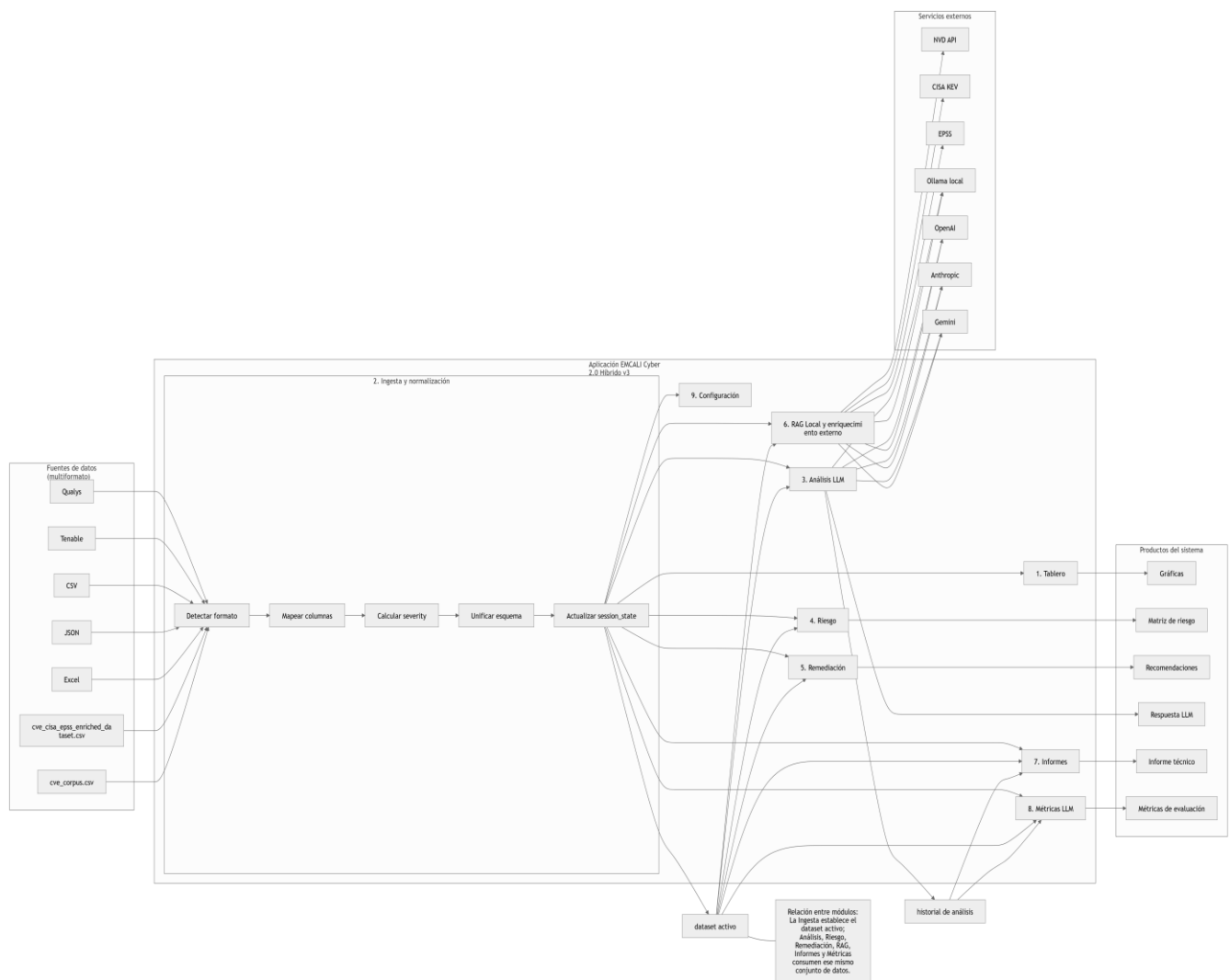


Figura 5. Diseño Topología funcional y módulos internos con ingesta de datos.

3.1 Lectura del modelo único de arquitectura

- Las fuentes de datos alimentan una capa de ingesta multiformato que detecta formato, mapea columnas, normaliza y unifica el esquema.
- El resultado de la ingesta se materializa en una base operativa de sesión compuesta por `findings_df`, `knowledge_base`, `analysis_history`, `reports_history` y `settings`.
- Los módulos de aplicación no conservan copias aisladas de datos: consultan el mismo estado compartido, asegurando consistencia operativa.
- La capa de inteligencia híbrida separa el selector de backend, el constructor de prompt, la recuperación RAG y el enriquecimiento externo.
- Las salidas reúnen priorización, matriz de riesgo, acciones de remediación, informe ejecutivo, métricas y JSON analítico.

3.2 Lectura del diagrama lógico por capas

El diagrama por capas explicita que la presentación y la navegación dependen de una capa de orquestación y aplicación; esta a su vez activa componentes de inteligencia y analítica, que consumen tanto la capa de modelos como la capa de datos y conocimiento. La capa de infraestructura y seguridad sirve de base para el entorno virtual Python, la gestión de claves API, el acceso controlado a internet y la compilación del ejecutable Windows.

4. Flujo operativo end-to-end

1. El usuario carga uno o varios archivos en Ingesta, o restablece el dataset demo.
2. La aplicación detecta el perfil del archivo (por ejemplo Qualys-like, Tenable-like o dataset enriquecido).
3. Se aplica el mapeo de columnas, cálculo de severity y exploitability, y se construye el esquema unificado.
4. El DataFrame normalizado reemplaza `findings_df` en `session_state` y se actualizan los metadatos de la última ingesta.
5. Tablero, Análisis, Riesgo, Remediación, RAG, Informes y Métricas consultan desde ese momento el mismo dataset activo.
6. Cuando el usuario ejecuta un análisis, el motor construye el prompt con contexto del hallazgo, evidencia RAG y, opcionalmente, enriquecimiento externo NVD.
7. La respuesta del backend LLM y la latencia se almacenan en `analysis_history`.
8. Riesgo, Informes y Métricas pueden consolidar resultados usando tanto el dataset activo como el historial analítico.

5. Ingesta multiformato y normalización avanzada

Uno de los cambios más relevantes de la versión v3 es la capacidad de admitir diferentes formatos de entrada sin exigir al usuario un esquema rígido. El normalizador implementa una detección heurística de perfiles y un conjunto de alias de columnas que permite convertir archivos operativos y enriquecidos a un modelo común.

5.1 Perfiles de ingesta soportados

Perfil detectado	Indicadores típicos	Tratamiento
operational_findings	finding_id, severity, asset, source	Se conserva el esquema casi sin transformación.
qualys_like	qid, vuln_type, severity, qds	Se mapean hallazgos Qualys al esquema unificado.
tenable_nessus_like	plugin_id, plugin_name, severity	Se asigna fuente Tenable/Nessus y se normaliza el score.
nvd_cisa_epss_enriched	cve + cvss + epss o cisa_kev	Se infieren severity, exploitability y recomendaciones por defecto.
cve_corpus	cve + description	Se usa el corpus como entrada descriptiva enriquecida.
generic_tabular	columnas tabulares parciales	Se aplica mapeo heurístico y se completan campos por defecto.

5.2 Reglas de derivación automática

- Si no existe severity y sí existe cvss, la severidad se deriva con rangos CVSS (Critical, High, Medium, Low).
- Si no existe exploitability, se estima con una función que combina CVSS, EPSS y presencia en CISA KEV.
- Si no existe asset, se construye a partir de vendor/product o se reutiliza el CVE como identificador sintético.
- Si no existe recommendation, se genera una acción base orientada a validación, priorización y remediación.
- Si no existe cisa_kev o epss, se generan valores por defecto para garantizar continuidad analítica.

5.3 Extracto de código del normalizador

```
COLUMN_ALIASES = {
    "cve_id": "cve",
    "baseScore": "cvss",
    "cvssScore": "cvss",
    "hostname": "asset",
    "ip_address": "ip",
    "vendorProject": "vendor",
    "product": "title",
}

if "severity" not in work.columns:
    work["severity"] = work["cvss"].apply(severity_from_cvss)

if "exploitability" not in work.columns:
    work["exploitability"] = work.apply(
        lambda row: exploitability_from_values(row.get("cvss"), row.get("epss"),
        row.get("cisa_kev")),
        axis=1,
    )
```

6. Encadenamiento por session_state y coherencia operativa

La corrección central de la v3 fue lograr que todos los módulos respondan al dataset activo cargado en Ingesta. Para ello, la aplicación usa `st.session_state` como base operativa compartida. Este patrón evita la duplicidad de DataFrames por módulo y asegura que una nueva carga se refleje inmediatamente en el tablero, las listas desplegadas, las gráficas y los reportes.

- `findings_df`: dataset activo normalizado.
- `knowledge_base`: base local de documentos y playbooks para RAG.
- `analysis_history`: historial de ejecuciones de análisis LLM con latencia, backend, salida y errores.
- `reports_history`: historial de informes generados.
- `settings`: parámetros de backend, modelo, prompts, timeout, claves presentes y enriquecimiento externo.

7. Descripción funcional de módulos

Módulo	Propósito	Entrada principal	Salida/efecto
Tablero	Vista ejecutiva del estado operativo.	<code>findings_df</code> activo y metadatos de ingesta.	KPI, gráficas por severidad y fuente, contexto del modelo híbrido.
Ingesta	Carga y normaliza archivos multiformato.	Archivos CSV, JSON, Excel o dataset demo.	Actualiza <code>findings_df</code> y <code>last_ingest_meta</code> .
Análisis LLM	Construye prompts y consulta el backend seleccionado.	Hallazgo seleccionado + prompt del analista + evidencia RAG.	Resumen ejecutivo, score, respuesta LLM, JSON y latencia.
Riesgo	Calcula prioridad y visualiza matriz de riesgo.	<code>findings_df</code> activo.	Scatter de riesgo, top hallazgos y tabla ordenada.
Remediación	Genera plan y script de validación.	Hallazgo seleccionado del dataset activo.	Secuencia de tratamiento y archivo descargable.
RAG Local	Explica y explora el contexto recuperado.	Consulta y hallazgo activo.	Pasajes locales, contexto NVD y explicación de relación entre módulos.
Informes	Produce salidas ejecutivas y descargables.	Dataset activo y estado analítico.	TXT, CSV, JSON y registro de reportes.
Métricas LLM	Evalúa cobertura y desempeño.	<code>findings_df</code> + <code>analysis_history</code> .	Indicadores, cobertura del dataset y latencia.
Configuración	Administra backends, prompts y secretos.	Parámetros de operación.	Settings activos y persistencia de API keys en <code>.env</code> .

8. Inteligencia híbrida y gestión de backends

La aplicación puede operar con cinco backends: mock, ollama, openai, anthropic y gemini. Esto permite adaptar la inferencia a la capacidad del equipo local. En portátiles sin GPU dedicada, el diseño favorece dos estrategias: usar un modelo local relativamente liviano como llama3.1:8b cuando se requiere operación offline o privacidad, y recurrir a proveedores cloud cuando se necesiten mejores tiempos de respuesta o razonamiento más amplio.

Backend	Modo de uso	Ventajas	Consideraciones
mock	Pruebas rápidas sin inferencia real.	Estabilidad y depuración de interfaz.	No genera respuesta analítica verdadera.
ollama	Inferencia local por API en <code>http://127.0.0.1:11434</code> .	Privacidad, continuidad offline, control del modelo.	Latencia dependiente del hardware local.
openai	Backend cloud vía API key en <code>.env</code> .	Alta calidad general y ecosistema maduro.	Requiere conectividad, credenciales y control de costo.
anthropic	Backend cloud orientado a respuestas extensas y estructuradas.	Buen desempeño en redacción analítica.	Requiere API key y conectividad.
gemini	Backend cloud de baja latencia relativa.	Respuesta ágil y útil para equipos limitados.	Requiere API key y acceso a internet.

8.1 Extracto del enrutador de backends

```
def call_backend(backend, prompt, model, settings):
    if backend == "mock":
        return call_mock(prompt, model)
    if backend == "ollama":
        return call_ollama(prompt, model, settings.get("ollama_base_url"),
settings.get("ollama_timeout_sec", 300))
    if backend == "openai":
        return call_openai(prompt, model, settings.get("openai_base_url"))
    if backend == "anthropic":
        return call_anthropic(prompt, model, settings.get("anthropic_base_url"))
    if backend == "gemini":
        return call_gemini(prompt, model, settings.get("gemini_base_url"))
    raise ValueError(f"Backend no soportado: {backend}")
```

9. RAG local y enriquecimiento externo

El módulo RAG Local no es una ventana aislada; constituye la explicación visible de un comportamiento transversal. La recuperación local consulta la base de conocimiento interna con playbooks, documentación, arquitectura y procedimientos de cierre. Cuando el hallazgo tiene un CVE válido y la opción está habilitada, también se solicita un resumen externo a NVD. Este contexto se reutiliza en el prompt de Análisis LLM y sirve de apoyo para priorización, remediación e informes.

- RAG local: evidencia contextual basada en documentos internos, playbooks y arquitectura.
- Enriquecimiento externo: resumen NVD y señales complementarias como CISA KEV y EPSS.

- Prompt builder: integra contexto del hallazgo, prompt del analista, plantilla del sistema, evidencia RAG y contexto externo.
- Impacto transversal: la misma evidencia alimenta análisis, riesgo, remediación, reportes y evaluación.

10. Motor analítico: priorización, riesgo y métricas

El cálculo del `priority_score` combina severidad, explotabilidad, criticidad del servicio, disponibilidad de parche, probabilidad de explotación (EPSS) y presencia en CISA KEV. Esta estrategia busca aproximar una lectura más operacional que el simple CVSS. La función `risk_matrix` deriva además `likelihood` e `impact_score` para construir la visualización tipo matriz de riesgo.

```
priority_score = CVSS × peso_severity × peso_explotability × bono_servicio × bono_parche
× bono_EPSS × bono_KEV
```

Las métricas del módulo LLM no se calculan contra un dataset fijo de demostración una vez que el usuario hace una nueva ingesta. En v3, la evaluación se hace respecto al dataset activo y al `analysis_history` de la sesión, lo que permite medir cobertura de hallazgos analizados, latencia y salidas exitosas del backend seleccionado.

10.1 Gráficas de referencia del prototipo

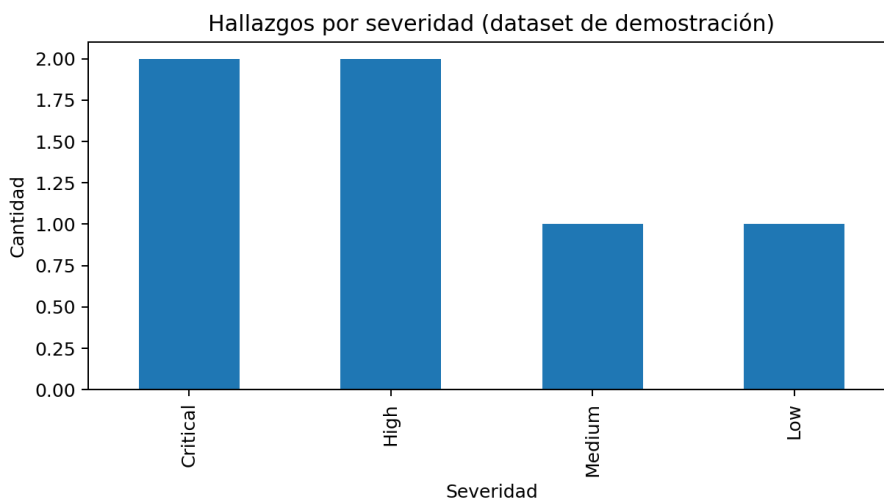


Figura 6. Hallazgos por severidad (dataset de demostración).

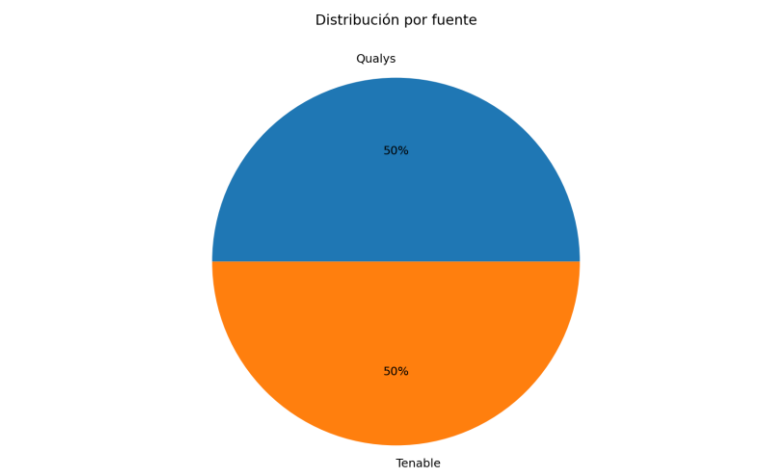


Figura 7. Distribución por fuente.

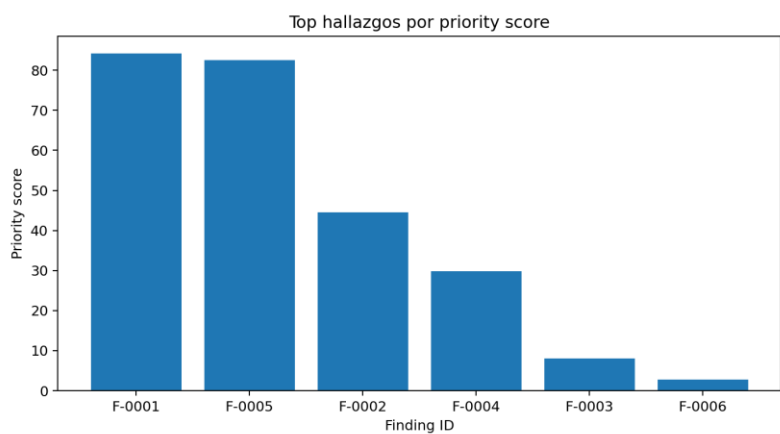


Figura 8. Top hallazgos por priority score.

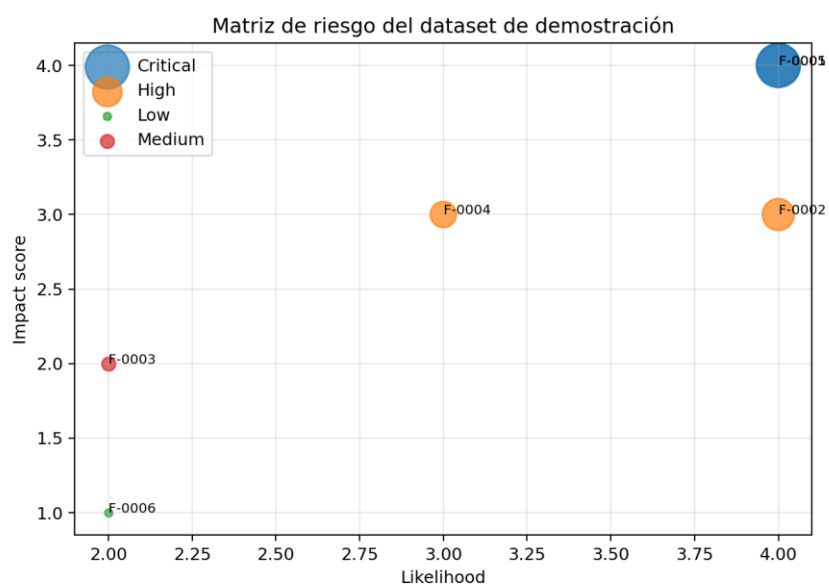


Figura 9. Matriz de riesgo del dataset de demostración.

11. Evidencia visual de la aplicación

Las siguientes capturas ilustran la configuración híbrida y la pantalla de análisis LLM de la aplicación. Estas vistas son relevantes porque muestran la parametrización del backend, el modelo activo, el timeout de inferencia y la interacción del analista con el prompt editable y el hallazgo seleccionado.

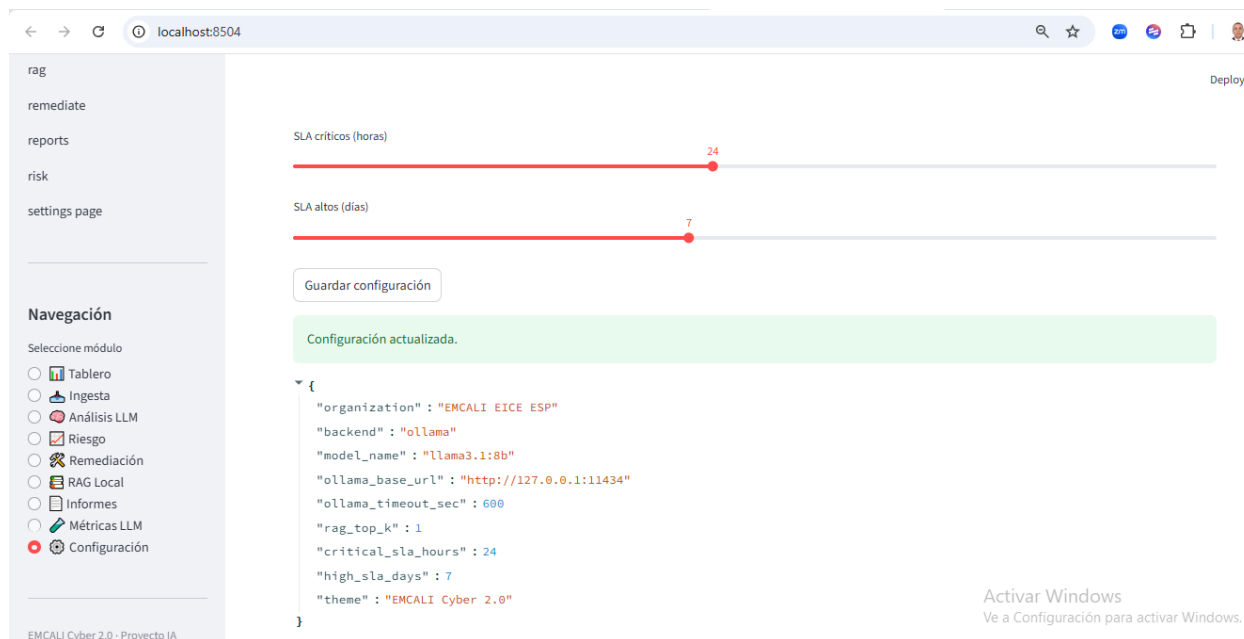


Figura 10. Pantalla de configuración con backend híbrido, timeout y parámetros de operación.

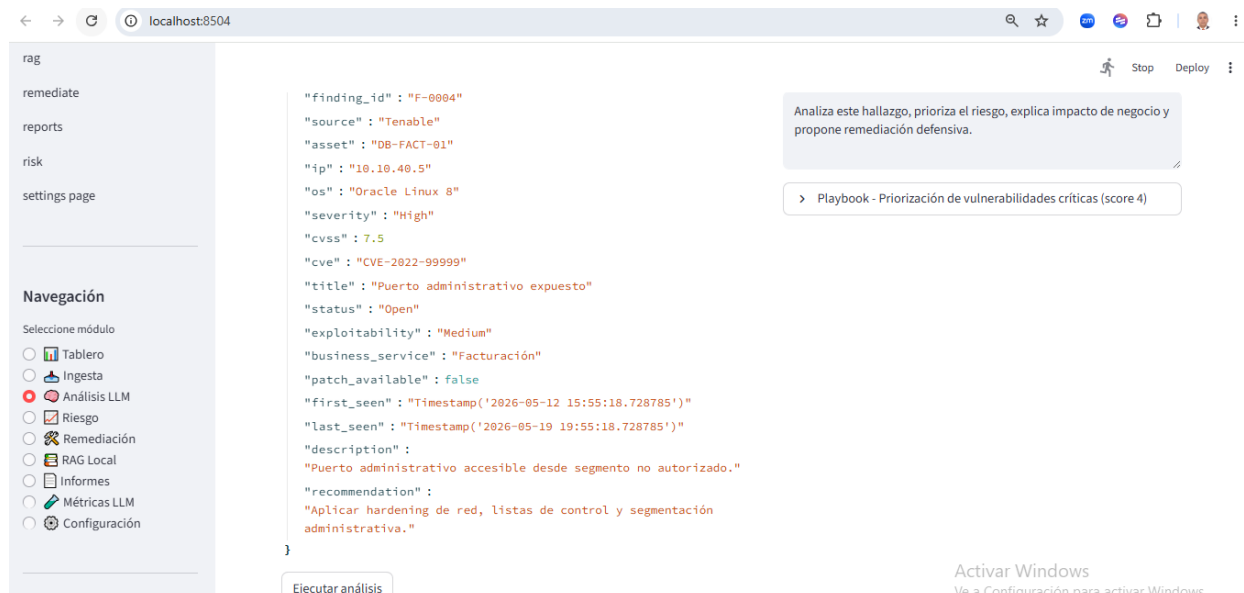


Figura 11. Pantalla de Análisis LLM con selección de hallazgo, prompt editable y contexto RAG.

12. Seguridad, claves API y operación controlada

La aplicación gestiona credenciales sensibles mediante archivo `.env` local y variables de entorno. El módulo de configuración permite registrar claves API de OpenAI, Anthropic y Gemini en ese archivo, reduciendo la necesidad de modificar código fuente. Para el backend local, la integración con Ollama se realiza mediante una URL configurable, normalmente `http://127.0.0.1:11434`. En todos los casos, el principio de seguridad es evitar la incrustación de secretos reales dentro del código distribuido.

- `python-dotenv` carga variables desde `.env` cuando están disponibles.
- El módulo `secrets_manager` actualiza el archivo `.env` y enmascara valores en la interfaz.
- Las API keys se consideran configuración local del equipo donde se ejecuta la aplicación.
- El acceso a internet se usa de forma controlada y no como navegación abierta indiscriminada.

13. Empaquetado y despliegue Windows

La solución se entregó también como kit de compilación Windows. El flujo de empaquetado usa PyInstaller para el ejecutable y NSIS para el Setup cuando está disponible. El entregable resultante contempla tres artefactos principales: un ejecutable en dist, un ZIP portable y un instalador Setup. Esta estrategia permite instalar la herramienta en equipos de demostración sin requerir el entorno de desarrollo completo.

- `build_windows_portable.bat` y `build_windows_portable.ps1` automatizan la compilación.
- `EMCALI_Cyber20_Hibrido.spec` define la recolección de archivos para PyInstaller.
- `create_nsis_installer.nsi` construye el instalador Setup.exe.
- `run_app.bat` facilita la ejecución en modo desarrollo con entorno virtual.

14. Limitaciones observadas y recomendaciones de mejora

- La inferencia local depende fuertemente del hardware. En equipos sin GPU, los tiempos con modelos grandes pueden ser altos.
- Los backends cloud dependen de credenciales válidas, acceso a internet y límites de cuota.
- El enriquecimiento externo actual es controlado y acotado; futuras versiones podrían ampliar el repertorio de fuentes con políticas de caching.
- La heurística de normalización cubre varios perfiles, pero siempre es posible refinar alias y reglas según nuevos datasets reales.
- El motor de métricas actual es útil para demostración y seguimiento, pero podría evolucionar hacia evaluación más rigurosa con datasets etiquetados y criterios comparativos.

15. Conclusiones

EMCALI Cyber 2.0 Híbrido v3 constituye una mejora sustancial respecto a las iteraciones previas, no solo por la cantidad de módulos disponibles, sino por la coherencia operativa conseguida entre ellos. La aplicación ya no depende de un dataset demo estático para el resto del flujo; la Ingesta se convierte en el punto de control del dataset activo y ese estado se propaga correctamente a todos los módulos. El diseño híbrido reduce la dependencia de un único backend y habilita escenarios locales, conectados o mixtos. La normalización multiformato mejora la utilidad del sistema para casos de datos reales. En conjunto, el prototipo ya presenta una base sólida para sustentación académica y para evolución hacia una herramienta más robusta de gestión de vulnerabilidades asistida por IA.

Anexo A. Estructura técnica del proyecto

```
EMCALI_Cyber20_Hibrido_v3_Source/
  .env.example
  CORRECCIONES_v3.md
  README.md
  app.py
  build_windows_desktop.py
  launcher.py
  requirements.txt
  requirements_desktop.txt
  run_app.bat
core/
  __init__.py
  analysis.py
  external_enrichment.py
  llm_clients.py
  normalizer.py
  sample_data.py
  secrets_manager.py
  state.py
  ui.py
  __pycache__/
pages/
  __init__.py
  analyze.py
  dashboard.py
  ingest.py
  metrics_eval.py
  rag.py
  remediate.py
  reports.py
  risk.py
  settings_page.py
  __pycache__/
assets/
packaging/
  windows/
    EMCALI_Cyber20_Hibrido.spec
    README_BUILD_WINDOWS.md
    build_windows_portable.bat
    build_windows_portable.ps1
    create_nsis_installer.nsi
    runtime_hook_path.py
    __pycache__/
  __pycache__/
```

Anexo B. Extractos clave de implementación

B.1 Inicialización del estado compartido

```
if "findings_df" not in st.session_state:
    st.session_state.findings_df = sample_findings_df()
...
if "settings" not in st.session_state:
    st.session_state.settings = {
        "backend": "mock",
        "model_name": "llama3.1:8b",
        "rag_top_k": 3,
        "enable_external_enrichment": True,
    }
```

B.2 Construcción del análisis y registro de latencia

```
result = analyze_finding(
    row=row,
    retrieved_docs=docs,
    settings=settings,
    user_query=query,
)
st.session_state.analysis_history.append({"finding_id": selected_id, **result})

if result.get("llm_response"):
    st.subheader("Respuesta del backend LLM")
    st.write(result["llm_response"])
```

B.3 Guardado de API keys en archivo .env

```
def save_env_key(key_name: str, key_value: str) -> None:
    ...
    ENV_PATH.write_text("
".join(new_lines) + "
", encoding="utf-8")
    os.environ[key_name] = key_value
    load_dotenv(override=True)
```

Referencias orientadoras

- Kaggle. Vulnerability Management Datasets: cve_cisa_epss_enriched_dataset.csv y cve_corpus.csv.
- NVD. National Vulnerability Database.
- CISA. Known Exploited Vulnerabilities Catalog.
- EPSS. Exploit Prediction Scoring System.
- Ollama. API local para modelos de inferencia on-premise.
- Documentación oficial de OpenAI, Anthropic y Gemini para integración por API.
- Documentación interna generada previamente para EMCALI Cyber 2.0, versiones modular, híbrida y v3 corregida.